

CS & IT ENGINEERING



Operating System

Process Synchronization

Lecture -7



By- Vishvadeep Gothi sir

Recap of Previous Lecture



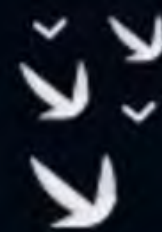
doubt?



Topic

Semaphore

Topics to be Covered



Topic

Classical Synchronization Problem



#Q. Given below is a program which when executed spawns two concurrent processes:

Semaphore X: ~~=0~~; ~~1~~ ~~0~~ ~~1~~ ~~0~~

/* Process now forks into concurrent processes P1 & P2 */

P1 : repeat forever

V(X);

→ Compute;

→ P(X);

P2: repeat forever

P(X);

→ Compute;

→ V(X);

Consider the following statements about processes P1 and P2:

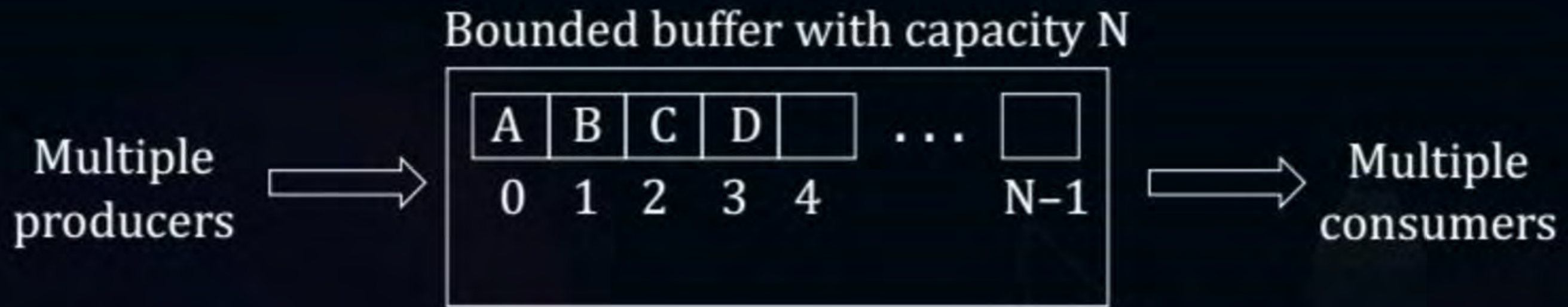
✓ I: It is possible for process P1 to starve.

✓ II. It is possible for process P2 to starve.



Topic : Bounded Buffer Problem

(Producer - Consumer)





Topic : Bounded Buffer Problem

- Known as producer-consumer problem also
- Buffer is the shared resource between producers and consumers





Topic : Bounded Buffer Problem : Solution

- Producers must block if the buffer is full
- Consumers must block if the buffer is empty





Topic : Bounded Buffer Problem : Solution

- **Variables:**

- Mutex: Binary Semaphore to take lock on buffer (Mutual Exclusion)
- Full: Counting Semaphore to denote the number of occupied slots in buffer
- Empty: Counting Semaphore to denote the number of empty slots in buffer

Initialization :- (initially buffer is Empty)

$\text{mutex} = 1$

$\text{Full} = 0$

$\text{Empty} = N$



Topic : Producer() & Consumer

Producer

```
wait(Empty)
wait(mutex)
//add item on buffer
signal(mutex)
signal(Full)
```

Consumer

```
wait(Full)
wait(mutex)
//remove item from buffer
signal(mutex)
signal(Empty)
```



Topic : Reader-Writer Problem

Consider a situation where we have a file shared between many people:

- If one of the people tries editing the file, no other person should be reading or writing at the same time, otherwise changes will not be visible to him/her
- However, if some person is reading the file, then others may read it at the same time

start ↓	Reader	writer
Reader	✓	✗
writer	✗	✗



Topic : Reader-Writer Problem Solution

- If writer is accessing the file, then all other readers and writers will be blocked
- If any reader is reading, then other readers can read but writer will be blocked



Topic : Reader-Writer Problem Solution

- **Variables:**
 - mutex: Binary Semaphore to provide Mutual Exclusion
 - wrt: Binary Semaphore to restrict readers and writers if writing is going on
 - Readcount: Integer variable, denotes number of active readers
- **Initialization:**
 - mutex: 1
 - w r t : 1
 - Read count: 0



Topic : Writer() & Reader() Processes

writer :-

wait(wrt)
// writing
signal(wrt)

Reader :-
wait(mutex)
Readcount ++;
if (Readcount == 1)
{ wait(wrt); }
signal(mutex)
// Reading
wait(mutex)
Readcount --;
if (Readcount == 0)
{ signal(wrt); }
signal(mutex)

wrt = ~~1~~ 0 ~~1~~ 0 1
RC = 0 ~~1~~ 2 ~~3~~ 2 ~~1~~ 0
mutex = ~~1~~ 0 ~~1~~ 0 ~~1~~
 0 1

- ex:- 1
1. writer $\Rightarrow$ writing ✓
 2. Reader $\Rightarrow$ stuck ✓
 3. Reader $\Rightarrow$ stuck at wait(mutex)
 4. Reader $\Rightarrow$



Topic : Dining Philosopher Problem





Topic : Dining Philosopher Problem

- K philosophers seated around a circular table
- There is one chopstick between each philosopher
- A philosopher may eat if he can pick up the two chopsticks adjacent to him
- One chopstick may be picked up by any one of its adjacent followers but not both



Topic : Dining Philosopher Problem Solution

Binary semaphore array for chopsticks $\Rightarrow ch[k] = \{1, 1, \dots, 1\}$

P_i

```
wait(ch[i])
wait(ch[(i+1)%k])
// eating
signal(ch[i])
signal(ch[(i+1)%k])
```




Topic : Characteristics of Semaphores

- Used to provide mutual exclusion
 - Used to control access to resources
 - Solution using semaphore can lead to have deadlock
 - Solution using semaphore can lead to have starvation
 - Solution using semaphore can be busy waiting solutions
 - Semaphores may lead to a priority inversion
 - Semaphores are machine-independent
- Higher priority process waits for lower priority process.



2 mins Summary



Topic

Classical Synchronization Problem





Happy Learning

THANK - YOU

